The Journal of Open Source Software

MRRpy: A modular Python model for distributed rainfall–runoff, flood routing, and inundation mapping

Saurav Bhattarai [1¶], **Nawa Raj Pradhan** [2], **and Rocky Talchabhadel** [1]

1 Department of Civil and Environmental Engineering, Jackson State University, Jackson, MS, USA
2 Coastal and Hydraulics Laboratory, U.S. Army Engineer Research and Development Center, Vicksburg, MS, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

MRRpy is an open-source, modular Python package that simulates floods, from rain to flood map, for a river basin anywhere on Earth. From a digital elevation model (DEM) and rainfall from a design storm, rain gauges, satellites, or weather-model forecasts, it traces the drainage network, estimates how much rain runs off, routes the water through every grid cell and river channel, and maps flood depth and extent along every river or, in two dimensions (2D), over a chosen town, river segment, or whole watershed. Every step offers interchangeable methods, and any combination runs from one configuration through a Python API, a command-line wizard, a Jupyter form (Kluyver et al., 2016), or a QGIS plugin (QGIS Development Team, 2026). Missing inputs are fetched from Google Earth Engine and Open-Meteo, so even a basin with no local data can be modeled, and its floods forecast, within minutes. MRRpy installs from PyPI and is documented at https://mrrpy.readthedocs.io.

Statement of need

Floods expose about 1.8 billion people to significant risk (Rentschler et al., 2022), many in basins with few or no gauges, where models must start from global data (Hrachowitz et al., 2013). Flood studies usually chain separate tools for rainfall, runoff, routing, and mapping, each with its own formats. Comparing two runoff methods with routing held fixed, testing a new infiltration model, or turning a weather forecast into a flood map means rebuilding that chain by hand, and inputs, mass accounting, and outputs are easily made inconsistent. MRRpy factors precipitation, runoff generation, routing, and flood mapping into independent stages behind one configuration, so a comparison is a change of setting and a new method inherits the rest of the model, its inputs, outputs, and tests. Because each stage hands the next only a field, users can bring their own physics: their own rainfall product (as gauge-style series) or runoff algorithm (a registered class, or runoff rasters from another model) drives MRRpy's routing and flood mapping unchanged, and a user who already has runoff and needs only a router can use those stages alone. With MRRpy, users can:

- **model an ungauged basin** from global terrain, soils, land cover, and satellite or weather-model rainfall, then swap in local data;
- **compare methods** with everything else held fixed, or sweep every combination of methods and channel geometry;
- **forecast floods** from weather-model rainfall up to 16 days ahead, continuing each update from the saved model state, or replay archived forecasts as hindcasts;
- **route known inflows** (a gauge record, a reservoir release, a dam-break or lake-outburst hydrograph) from any river point, with or without rain;
- **map floods** along every river in seconds, or in 2D over a town, several river segments, or a whole watershed.

DRAFT

MRRpy serves hydrologists, engineers, and forecasters; developers testing new components; GIS practitioners through its QGIS plugin; and students, through readable code and an interactive textbook (https://sauravbhattarai19.github.io/MRRpy/).

State of the field

pysheds (Bartos, 2020) and pyflwdir (Eilander & Boisgontier, 2021) are strong Python libraries for D8 terrain analysis but stop at flow directions and delineation; MRRpy uses either and continues through runoff, routing, and flood mapping. Landlab (Barnhart et al., 2020) is a general Earth-surface-dynamics framework from whose components a comparable model could be assembled, with more effort. GSSHA (Downer & Ogden, 2004) is a mature, physically based distributed hydrologic model under continued development (Pradhan et al., 2019; Pradhan & Floyd, 2024), while LISFLOOD-FP (Bates et al., 2010; de Almeida et al., 2012) and HEC-RAS (Brunner, 2021) provide advanced floodplain hydraulic modeling capabilities, and Height Above Nearest Drainage (HAND) methods (Nobre et al., 2011; Zheng et al., 2018) map the U.S. National Water Model's forecasts (Aristizabal et al., 2023). Complementing these established modeling systems, MRRpy emphasizes a modular Python-based architecture that facilitates the integration, interchange, and evaluation of runoff-generation, routing, and flood-mapping methods through standardized interfaces. We are not aware of an open tool that lets a user combine any rainfall source, runoff method, and routing scheme on the same grid, add a new method that immediately works with all the others, check every combination for mass conservation, and carry the result to a flood map. For engineering design around levees, culverts, or bridges, a dedicated hydraulic model remains the right tool.

Software design

One configuration object drives the pipeline (Figure 1) from every interface, and one parameter catalog generates the wizard, the notebook form, and the manual's tables.

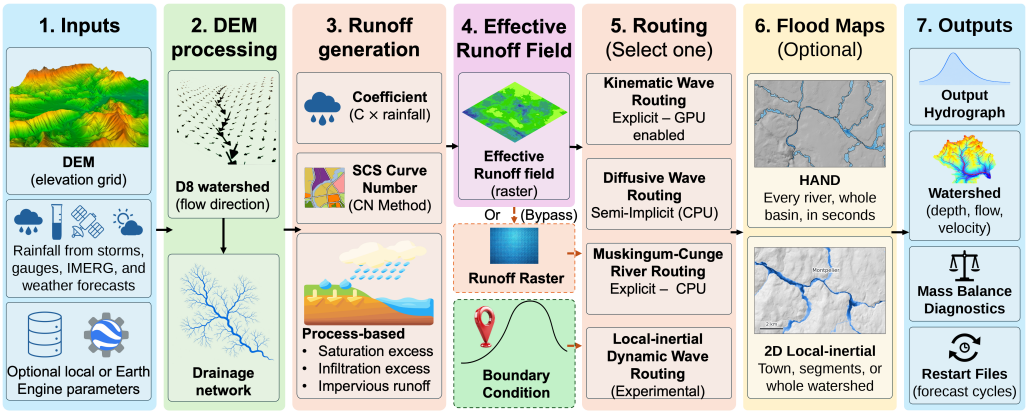


Figure 1: MRRpy's processing chain, from terrain and precipitation (including weather-model forecasts) through selectable runoff and routing methods and optional flood maps (HAND, or 2D over a town, river segments, or the whole watershed) to the outputs, including restart files for forecast updates. Flood-map panels: MRRpy results for Kathmandu, Nepal (HAND), and Montpelier, Vermont (2D).

Terrain and parameters. DEM processing conditions the terrain (by default least-cost carving (Yamazaki et al., 2012) with pyflwdir (Wang & Liu, 2006); pysheds (O'Callaghan & Mark, 1984) with filling remains available), computes D8 flow directions, and delineates the catchment above an outlet or keeps the whole DEM. The DEM, soil, and land-cover parameters come

DRAFT

from local inputs or Earth Engine (Gorelick et al., 2017), including ESA WorldCover (Zanaga et al., 2022), SoilGrids (Poggio et al., 2021), and HiHydroSoil (Simons et al., 2020).

Precipitation returns a rain-rate field per cell at any time. Sources are a uniform hyetograph, rain gauges (Thiessen polygons or inverse-distance weighting), IMERG satellite rainfall (Huffman et al., 2020), and Open-Meteo (Zippenfenig, 2023) weather-model reanalyses, forecasts, and archived forecast runs; satellite pixels and model cells become pseudo-gauges for the same interpolators.

Runoff generation is a registry of interchangeable modes behind one interface: direct rainfall, a runoff coefficient, SCS Curve Number (Chow et al., 1988; Jaafar et al., 2019), a prescribed raster, and a process-based mode. The process-based mode is itself a second registry of mechanisms composed per cell without counting rainfall twice; the built-in ones are Green–Ampt infiltration excess (Green & Ampt, 1911; Rawls et al., 1983), saturation excess, and impervious shedding, with soil recovery between storms following EPA SWMM 5 (Rossman & Huber, 2016). Saturation excess uses the variable-source-area theory of Pradhan & Ogden (2010), which needs only an initial soil-moisture deficit, here from the satellite-based SERVES method (Pradhan, 2019); developed earlier as a stand-alone theory, it is applied here for distributed simulation alongside standard Green–Ampt. A new method is a class registered with a decorator.

Routing schemes are likewise registered. Kinematic wave, variable-parameter Muskingum–Cunge (Cunge, 1969; Ponce & Yevjevich, 1978), and a semi-implicit diffusive wave are supported, and a local-inertial scheme (de Almeida et al., 2012) is experimental. Channel cells carry a sub-grid channel (Neal et al., 2012) sized to a bankfull discharge that is user-given or estimated globally (Andreadis et al., 2013; Linke et al., 2019). The semi-implicit scheme applies the classical direct elimination for tree-structured systems to the whole D8 grid, giving an exact implicit solve in linear time without the explicit Courant limit, using Numba (Lam et al., 2015) or SciPy (Virtanen et al., 2020). Kinematic routing, runoff, and precipitation can run on a CuPy (Okuta et al., 2017) GPU backend. Inflow hydrographs can enter at any channel point, with or without rain.

Flood maps. Routing keeps overbank water in a D8 ribbon one cell wide, so an optional stage spreads it over the land. Maps are drawn after routing from the saved discharge, on a finer DEM where one exists (such as LiDAR or 30 m FABDEM (Hawker et al., 2022)), so they can be redrawn without rerouting and 2D effort goes only where needed. HAND mapping (Nobre et al., 2011) turns each reach's flow into a water level with a synthetic rating curve (Zheng et al., 2018) and floods all land below it: every river in seconds, but blind to backwater. The 2D option solves the local-inertial equations (Bates et al., 2010) over a town, several river segments, or the whole watershed, each fed by the routed rivers and inflow, so water spreads, ponds, and backs up while mass is conserved exactly.

Outputs are common to every combination: an outlet hydrograph, a mass-balance ledger, and optionally virtual gauges, per-cell fields, runoff by mechanism, and flood depth, extent, and timing maps. Restart files carry the model state between forecast updates, reproducing an uninterrupted run.

Testing. Because every method meets the same interface, the model is tested as a whole. Continuous integration (Python 3.9–3.12) runs each supported routing scheme with each runoff method and every subset of the physical mechanisms over two storms separated by a dry spell; each run must close its mass balance to 10^{-6} and never produce more runoff than rain, so a new component is checked against every existing one. Further tests compare restarted with uninterrupted runs and GPU with CPU, check the 2D solver against a lake at rest and an analytic flood wave (N. M. Hunter et al., 2005), and verify the semi-implicit solve against direct solvers.

DRAFT

Research impact statement

MRRpy 1.0 is released on PyPI with a QGIS plugin, a manual, 13 worked examples, Jupyter notebooks, and an interactive textbook, so each capability can be reproduced. Several examples simulate historical floods in Nepal, the United States, and Germany from global data alone.

MRRpy drives an experimental, automated flood forecast for Lynch Creek, a 38 km^2 urban basin in Jackson, Mississippi (https://github.com/SauravBhattarai19/lynch-forecast). Every six hours it takes rainfall forecasts from five weather models, continues the model from its saved state, maps flood depth along every creek on 1 m lidar, and publishes a public dashboard (https://sauravbhattarai19.github.io/lynch-forecast/); the same setup maps the 10- and 20-year return-period floods (NOAA Atlas 14 design storms) in 2D over four river segments. All of its hydrology comes from MRRpy, so another basin needs only its own configuration files. More applications will follow as worked examples.

Its significance is practical. Once a basin is set up, the same model can be calibrated against gauges by sweeping parameters such as roughness or soil conductivity, run for many scenarios (design storms, land-use change, or upstream, reservoir-release, and dam-break hydrographs as boundary conditions), and tracked at any point through virtual gauges and flood maps. Driven by weather-model forecasts, it maps the coming inundation before the flood arrives, so early warning and flood-risk planning become feasible in basins with little data, and a shared interface spares method research new plumbing.

AI usage disclosure

Anthropic's Claude Code CLI (Claude Sonnet 4.6, Opus 4.6, Opus 4.8, Sonnet 5, Fable 5, and Opus 5.5) was used for code implementation, refactoring, test scaffolding, documentation, and drafting and revising this manuscript. All AI-assisted code and text were reviewed, tested, and edited by the human authors, who are responsible for the correctness of the software, the validity of the reported results, and the content of this paper.

Acknowledgements

MRRpy is built on the open-source scientific-Python and geospatial ecosystem, including NumPy (Harris et al., 2020), SciPy (Virtanen et al., 2020), pandas (McKinney, 2010; The pandas development team, 2020), Matplotlib (J. D. Hunter, 2007), `rasterio` (Gillies & Rasterio contributors, 2013), GeoPandas (Fleischmann et al., 2026), `pysheds`, and `pyflwdir`, and on the Google Earth Engine (Gorelick et al., 2017) and Open-Meteo (Zippenfenig, 2023) data services. We thank their developers and maintainers.

This research was supported in part by an appointment to the Department of Defense (DOD) Research Participation Program administered by the Oak Ridge Institute for Science and Education (ORISE) through an interagency agreement between the U.S. Department of Energy (DOE) and the DOD. ORISE is managed by ORAU under DOE contract number DE-SC0014664. All opinions expressed in this paper are the authors' and do not necessarily reflect the policies and views of DOD, DOE, or ORAU/ORISE.

This research was also supported by the U.S. Army Engineer Research and Development Center, Coastal and Hydraulics Laboratory (ERDC–CHL), through its Earth System Hydrology research initiative, with technical support provided under the ERDC–CHL professional services contract with Taylor–Woolpert, LLC.

MRRpy, an open-source hydrologic modeling software package, was developed through research collaboration between the U.S. Army Engineer Research and Development Center, Coastal and Hydraulics Laboratory (ERDC–CHL), and the Hydrology Laboratory at Jackson State University (JSU).

DRAFT

References

Andreadis, K. M., Schumann, G. J.-P., & Pavelsky, T. (2013). A simple global river bankfull width and depth database. *Water Resources Research*, *49*(10), 7164–7168. https://doi.org/10.1002/wrcr.20440

Aristizabal, F., Salas, F., Petrochenkov, G., Grout, T., Avant, B., Bates, B., Spies, R., Chadwick, N., Wills, Z., & Judge, J. (2023). Extending height above nearest drainage to model multiple fluvial sources in flood inundation mapping applications for the U.S. National Water Model. *Water Resources Research*, *59*(5), e2022WR032039. https://doi.org/10.1029/2022WR032039

Barnhart, K. R., Hutton, E. W. H., Tucker, G. E., Gasparini, N. M., Istanbulluoglu, E., Hobley, D. E. J., Lyons, N. J., Mouchene, M., Nudurupati, S. S., Adams, J. M., & Bandaragoda, C. (2020). Short communication: Landlab v2.0: A software package for Earth surface dynamics. *Earth Surface Dynamics*, *8*(2), 379–397. https://doi.org/10.5194/esurf-8-379-2020

Bartos, M. (2020). *pysheds: Simple and fast watershed delineation in Python*. https://doi.org/10.5281/zenodo.3822494

Bates, P. D., Horritt, M. S., & Fewtrell, T. J. (2010). A simple inertial formulation of the shallow water equations for efficient two-dimensional flood inundation modelling. *Journal of Hydrology*, *387*(1–2), 33–45. https://doi.org/10.1016/j.jhydrol.2010.03.027

Brunner, G. W. (2021). *HEC-RAS River Analysis System hydraulic reference manual, version 6.0*. US Army Corps of Engineers, Hydrologic Engineering Center.

Chow, V. T., Maidment, D. R., & Mays, L. W. (1988). *Applied hydrology*. McGraw-Hill.

Cunge, J. A. (1969). On the subject of a flood propagation computation method (Muskingum method). *Journal of Hydraulic Research*, *7*(2), 205–230. https://doi.org/10.1080/00221686909500264

de Almeida, G. A. M., Bates, P., Freer, J. E., & Souvignet, M. (2012). Improving the stability of a simple formulation of the shallow water equations for 2-D flood modeling. *Water Resources Research*, *48*(5), W05528. https://doi.org/10.1029/2011WR011570

Downer, C. W., & Ogden, F. L. (2004). GSSHA: Model to simulate diverse stream flow producing processes. *Journal of Hydrologic Engineering*, *9*(3), 161–174. https://doi.org/10.1061/(ASCE)1084-0699(2004)9:3(161)

Eilander, D., & Boisgontier, H. (2021). *pyflwdir: Fast methods to work with hydro- and topography data in pure Python*. https://doi.org/10.5281/zenodo.5534755

Fleischmann, M., Van den Bossche, J., Jordahl, K., Richards, M. J., McBride, J., Wasserman, J., Ward, B., & Wolf, L. J. (2026). GeoPandas: Fundamental data structures for vector spatial data in Python. *Computers, Environment and Urban Systems*, *130*, 102495. https://doi.org/10.1016/j.compenvurbsys.2026.102495

Gillies, S., & Rasterio contributors. (2013). *Rasterio: Geospatial raster I/O for Python programmers*. https://github.com/rasterio/rasterio

Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). Google Earth Engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, *202*, 18–27. https://doi.org/10.1016/j.rse.2017.06.031

Green, W. H., & Ampt, G. A. (1911). Studies on soil phyics: 1. The flow of air and water through soils. *The Journal of Agricultural Science*, *4*(1), 1–24. https://doi.org/10.1017/S0021859600001441

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with

DRAFT

NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hawker, L., Uhe, P., Paulo, L., Sosa, J., Savage, J., Sampson, C., & Neal, J. (2022). A 30 m global map of elevation with forests and buildings removed. *Environmental Research Letters*, *17*(2), 024016. https://doi.org/10.1088/1748-9326/ac4d4f

Hrachowitz, M., Savenije, H. H. G., Blöschl, G., McDonnell, J. J., Sivapalan, M., Pomeroy, J. W., Arheimer, B., Blume, T., Clark, M. P., Ehret, U., Fenicia, F., Freer, J. E., Gelfan, A., Gupta, H. V., Hughes, D. A., Hut, R. W., Montanari, A., Pande, S., Tetzlaff, D., … Cudennec, C. (2013). A decade of Predictions in Ungauged Basins (PUB)—a review. *Hydrological Sciences Journal*, *58*(6), 1198–1255. https://doi.org/10.1080/02626667.2013.803183

Huffman, G. J., Bolvin, D. T., Braithwaite, D., Hsu, K.-L., Joyce, R., Kidd, C., Nelkin, E. J., Sorooshian, S., Tan, J., & Xie, P. (2020). *NASA Global Precipitation Measurement (GPM) Integrated Multi-satellitE Retrievals for GPM (IMERG)* [Algorithm Theoretical Basis Document (ATBD), version 06]. NASA.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Hunter, N. M., Horritt, M. S., Bates, P. D., Wilson, M. D., & Werner, M. G. F. (2005). An adaptive time step solution for raster-based storage cell modelling of floodplain inundation. *Advances in Water Resources*, *28*(9), 975–991. https://doi.org/10.1016/j.advwatres.2005.03.007

Jaafar, H. H., Ahmad, F. A., & El Beyrouthy, N. (2019). GCN250, new global gridded curve numbers for hydrologic modeling and design. *Scientific Data*, *6*(1), 145. https://doi.org/10.1038/s41597-019-0155-x

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., Willing, C., & Jupyter Development Team. (2016). Jupyter notebooks – a publishing format for reproducible computational workflows. In F. Loizides & B. Schmidt (Eds.), *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87–90). IOS Press. https://doi.org/10.3233/978-1-61499-649-1-87

Lam, S. K., Pitrou, A., & Seibert, S. (2015). Numba: A LLVM-based Python JIT compiler. *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 1–6. https://doi.org/10.1145/2833157.2833162

Linke, S., Lehner, B., Ouellet Dallaire, C., Ariwi, J., Grill, G., Anand, M., Beames, P., Burchard-Levine, V., Maxwell, S., Moidu, H., Tan, F., & Thieme, M. (2019). Global hydro-environmental sub-basin and river reach characteristics at high spatial resolution. *Scientific Data*, *6*, 283. https://doi.org/10.1038/s41597-019-0300-6

McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in science conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a

Neal, J., Schumann, G., & Bates, P. (2012). A subgrid channel model for simulating river hydraulics and floodplain inundation over large and data sparse areas. *Water Resources Research*, *48*(11). https://doi.org/10.1029/2012WR012514

Nobre, A. D., Cuartas, L. A., Hodnett, M., Rennó, C. D., Rodrigues, G., Silveira, A., Waterloo, M., & Saleska, S. (2011). Height above the nearest drainage – a hydrologically relevant new terrain model. *Journal of Hydrology*, *404*(1–2), 13–29. https://doi.org/10.1016/j.jhydrol.2011.03.051

O'Callaghan, J. F., & Mark, D. M. (1984). The extraction of drainage networks from digital elevation data. *Computer Vision, Graphics, and Image Processing*, *28*(3), 323–344.

DRAFT

https://doi.org/10.1016/S0734-189X(84)80011-0

Okuta, R., Unno, Y., Nishino, D., Hido, S., & Loomis, C. (2017). CuPy: A NumPy-compatible library for NVIDIA GPU calculations. *Proceedings of Workshop on Machine Learning Systems (LearningSys) in the Thirty-First Annual Conference on Neural Information Processing Systems (NIPS)*. http://learningsys.org/nips17/assets/papers/paper_16.pdf

Poggio, L., de Sousa, L. M., Batjes, N. H., Heuvelink, G. B. M., Kempen, B., Ribeiro, E., & Rossiter, D. (2021). SoilGrids 2.0: Producing soil information for the globe with quantified spatial uncertainty. *SOIL*, *7*(1), 217–240. https://doi.org/10.5194/soil-7-217-2021

Ponce, V. M., & Yevjevich, V. (1978). Muskingum-Cunge method with variable parameters. *Journal of the Hydraulics Division*, *104*(12), 1663–1667. https://doi.org/10.1061/JYCEAJ.0005119

Pradhan, N. R. (2019). Estimating growing-season root zone soil moisture from vegetation index-based evapotranspiration fraction and soil properties in the Northwest Mountain region, USA. *Hydrological Sciences Journal*, *64*(7), 771–788. https://doi.org/10.1080/02626667.2019.1593417

Pradhan, N. R., Downer, C. W., & Marchenko, S. (2019). Catchment hydrological modeling with soil thermal dynamics during seasonal freeze-thaw cycles. *Water*, *11*(1), 116. https://doi.org/10.3390/w11010116

Pradhan, N. R., & Floyd, I. (2024). Examining the effect of moisture thresholds on post-fire water-repellent soil: A large-scale modelling approach applied to the Upper Arroyo Seco watershed, California, USA. *International Journal of Wildland Fire*, *33*(4), WF22083. https://doi.org/10.1071/WF22083

Pradhan, N. R., & Ogden, F. L. (2010). Development of a one-parameter variable source area runoff model for ungauged basins. *Advances in Water Resources*, *33*(5), 572–584. https://doi.org/10.1016/j.advwatres.2010.03.002

QGIS Development Team. (2026). *QGIS Geographic Information System*. Open Source Geospatial Foundation Project. https://www.qgis.org

Rawls, W. J., Brakensiek, D. L., & Miller, N. (1983). Green-Ampt infiltration parameters from soils data. *Journal of Hydraulic Engineering*, *109*(1), 62–70. https://doi.org/10.1061/(ASCE)0733-9429(1983)109:1(62)

Rentschler, J., Salhab, M., & Jafino, B. A. (2022). Flood exposure and poverty in 188 countries. *Nature Communications*, *13*(1), 3527. https://doi.org/10.1038/s41467-022-30727-4

Rossman, L. A., & Huber, W. C. (2016). *Storm Water Management Model reference manual volume I – hydrology (revised)* (No. EPA/600/R-15/162A). U.S. Environmental Protection Agency.

Simons, G. W. H., Koster, R., & Droogers, P. (2020). *HiHydroSoil v2.0 – a high resolution soil map of global hydraulic properties* (FutureWater Report No. 213). FutureWater. https://www.futurewater.nl/wp-content/uploads/2020/10/HiHydroSoil-v2.0-High-Resolution-Soil-Maps-of-Global-Hydraulic-Properties.pdf

The pandas development team. (2020). *pandas-dev/pandas: Pandas*. Zenodo. https://doi.org/10.5281/zenodo.3509134

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wang, L., & Liu, H. (2006). An efficient method for identifying and filling surface

DRAFT

depressions in digital elevation models for hydrologic analysis and modelling. *International Journal of Geographical Information Science*, *20*(2), 193–213. https://doi.org/10.1080/13658810500433453

Yamazaki, D., Baugh, C. A., Bates, P. D., Kanae, S., Alsdorf, D. E., & Oki, T. (2012). Adjustment of a spaceborne DEM for use in floodplain hydrodynamic modeling. *Journal of Hydrology*, *436–437*, 81–91. https://doi.org/10.1016/j.jhydrol.2012.02.045

Zanaga, D., Van De Kerchove, R., Daems, D., De Keersmaecker, W., Brockmann, C., Kirches, G., Wevers, J., Cartus, O., Santoro, M., Fritz, S., Lesiv, M., Herold, M., Tsendbazar, N.-E., Xu, P., Ramoino, F., & Arino, O. (2022). *ESA WorldCover 10 m 2021 v200*. https://doi.org/10.5281/zenodo.7254221

Zheng, X., Tarboton, D. G., Maidment, D. R., Liu, Y. Y., & Passalacqua, P. (2018). River channel geometry and rating curve estimation using height above the nearest drainage. *Journal of the American Water Resources Association*, *54*(4), 785–806. https://doi.org/10.1111/1752-1688.12661

Zippenfenig, P. (2023). *Open-Meteo.com weather API*. Zenodo. https://doi.org/10.5281/zenodo.7970649

DRAFT